

Question (from image_ef591c.png):

- Analyze how the high cost of a page fault influences the design decisions of virtual memory systems. How can you solve it? / How can you reduce page fault? Explain logically. / Page fault → How to reduce

Answer:

- **The Problem (High Cost):** A page fault is extremely slow because accessing the disk takes millions of CPU cycles, making it about $100,000\times$ slower than RAM. This high cost forces virtual memory systems to be designed entirely around minimizing these faults. PDF + 1
- **The Solutions (How to Reduce/Solve It):**
 1. **Larger Page Sizes:** Using bigger pages (like 32 KB or 64 KB instead of 4 KB) reduces the total number of trips to the disk. PDF
 2. **Smart Replacement (LRU):** Using the Least Recently Used policy kicks out the oldest unused pages to keep active data in RAM. PDF
 3. **Fully Associative Placement:** Allowing pages to go anywhere in RAM removes fixed-slot conflict faults. PDF
 4. **Write-Back Scheme:** Only writing dirty data back to disk during page replacement avoids the extreme slowness of updating the disk on every single write. PDF

Question (from image_ef599a.png):

Q8: What is a Page Fault, and how does the system handle it?

Answer:

- **What it is:** A page fault occurs when a program tries to access a memory block (page) that is not currently loaded in the RAM and must be fetched from the disk. PDF
- **How the system handles it:**
 1. **Pause:** The OS temporarily pauses the execution of the program. PDF
 2. **Load:** The OS fetches and loads the missing page from the disk into the RAM. PDF
 3. **Update:** The OS updates the page table with the page's new location in RAM. PDF
 4. **Resume:** The OS resumes the program, restarting the instruction that caused the fault. PDF